12
Java-Scala-Prolog integration

Mirko Viroli
mirko.viroli@unibo.it

C.D.L. Magistrale in Ingegneria e Scienze Informatiche
Alma Mater Studiorum—Università di Bologna, Cesena

a.a. 2025/2026

Outline

1. On Java-Scala-Prolog Integration
2. Prolog calling Java
3. Java calling Prolog
4. Prolog libraries written in Java
5. Scala using Prolog

Advantages of Prolog

- Declarative style of programming
- Concise way of expressing algorithms and data structures
- Intrinsic search & re-try mechanisms
- Rapid prototyping
- (Meta-)Interpretation facilities
- (More near to model abstract concepts)

Advantages of Java (and Scala)

With Java

- Imperative style of programming, + OOP
- More easy for the typical programmer
- More efficient than Prolog
- Wide availability of libraries
  - Graphics, Network, File-System, XML, ...
- Declarativity added with functional support (Java 8+)

And with Scala

- Good Java integration
- Full support to FP
- For comprehension somewhat addresses space-search mechanisms
- Declarativity not yet reaches that of LP

Why Java-Scala-Prolog integration

The proposed scenario

- Using Java/Scala for aspects "in-the large" and interactions with OS
- Using Prolog for some data & behaviour of the core system

Examples

- When writing an agent, modelling its mind in Prolog
- In a game, computer strategy in Prolog
- Using Prolog DBs for storing data and knowledge

Integration kinds: Language A using language B

Library-oriented integration

- A library for A allows to exploit language B
- A handy approach, but the additional code can be complicated

Language-oriented integration

- Language A is extended with abstractions of language B
- The resulting language is often too much complicated

Encapsulation-oriented integration

- Code in language B can be encapsulated into abstractions of A
- Nice trade-off between the two above solutions

Modern approach (not used in tuProlog): DSL-embedding integration

- Code in language B can be used by a DSL that is actually an API
- Another nice trade-off between the two above solutions

Java-tuProlog(4.0) Integration technologies

Library-oriented integration

- Prolog using Java: `->/2`, `new_object/3`, `returns/2`
- Java using Prolog: `Term` hierarchy, `Engine` class

Encapsulation-oriented integration

- Prolog using Java: a Java class becomes Prolog library
- Scala using Prolog: a Prolog theory embedded into a Scala program

Outline

1 On Java-Scala-Prolog Integration

2 Prolog calling Java

3 Java calling Prolog

4 Prolog libraries written in Java

5 Scala using Prolog

1) Prolog using Java: library approach

Library-oriented integration

- Prolog using Java: `->/2`, `new_object/3`, `returns/2`
- Java using Prolog: `Term` hierarchy, `Engine` class

Encapsulation-oriented integration

- Prolog using Java: a Java class becomes Prolog library
- Scala using Prolog: a Prolog theory embedded into a Scala program

Example Java code

```
public class C{
    public static void main(String[] s){
        javax.swing.JFrame f=new javax.swing.JFrame();
        f.setSize(100,100);
        f.setVisible(true);
    }
}
```

- This is a simple case of a Java class that creates a window on the screen using `javax.swing` library

Equivalent Prolog code

```
createframe(Name,X,Y):-
        new_object('javax.swing.JFrame',[],Obj),
        Obj <- setTitle(Name),
        Obj <- setSize(X,Y),
        Obj <- setVisible(true).
```

```
?- createframe('Prova',400,300).
```

- Invoking the above goal causes a similar execution

Predicate new_object/3

Signature

- `new_object(+ClassName,+ArgumentList,-ObjRef)`
- `ClassName` is the package-qualified name
  - Retrieved through the classpath of the 2P application
- `ArgumentList` is a list of arguments for the constructor
  - integers, strings, Java objects (see below)
- `ObjRef` is a Prolog term used as Object reference
  - it is of the kind `'$obj_2'`
- It calls the Java construct of `ClassName` class, passing arguments, and the result is associated to handler `ObjRef`

Example

```
?- new_object('java.lang.Integer',[1],X).
```

Predicate <-/2

Signature

- `ObjRef <- MethName(Args)`
- It calls on `ObjRef` (obtained through `new_object`) method `MethName`, passing `Args` as arguments
- This works if we do not need any result

Example

```
?- new_object('javax.swing.JFrame',[],Obj),
       Obj <- setSize(X,Y),
       Obj <- setVisible(true).
```

Predicate returns/2

Signature

- `ObjRef <- MethName(Args) returns ObjRef2`
- `ObjRef2` unifies with the result

Example

```
?- new_object('javax.swing.JFrame',[],Obj),
        Obj <- setSize(X,Y),
        Obj <- setVisible(true),
        Obj <- isVisible returns Z.
Z/true
```

Other predicates

Example

- To read/write the field of an object
- To create/access Java arrays
- To load a class dynamically
- ...

Where/how learning more?

- See `tuProlog` documentation
- Do some experiments if you like

An example application: handling vectors

Want to handle vectors of integers in Prolog

Providing the following predicates

- `vector_new(-VectorRef)`
- `vector_add(+VectorRef,+Element)`
- `vector_addall(+VectorRef,+OtherVectorRef)`
- `vector_clear(+VectorRef)`
- `vector_contains(+VectorRef,+Element)`
- `vector_lookup(+VectorRef,+Position,-Element)`
- `vector_isempty(+VectorRef)`
- `vector_size(+VectorRef,-Size)`
- `vector_print(+VectorRef)`

Some examples

Code

```
vector_new(V):-new_object('java.util.Vector',[],V).
vector_add(V,E):-new_object('java.lang.Integer',[E],I),
                 V <- add(I).
vector_size(V,S):-V <- size returns S.
vector_lookup(V,P,E):-V <- elementAt(P) returns E.
```

Application: handling fast arrays in Prolog

```
?-vector_new(X),
  vector_add(X,1),
  vector_add(X,2),
  vector_lookup(X,0,N).
```

Outline

1. On Java-Scala-Prolog Integration
2. Prolog calling Java
3. Java calling Prolog
4. Prolog libraries written in Java
5. Scala using Prolog

2) Java using Prolog: library approach

Library-oriented integration

- Prolog using Java: `->/2`, `new_object/3`, `returns/2`
- Java using Prolog: `Term` hierarchy, `Engine` class

Encapsulation-oriented integration

- Prolog using Java: a Java class becomes Prolog library
- Scala using Prolog: a Prolog theory embedded into a Scala program

The tuprolog Java library

The approach..

Via an API providing:

- Classes representing an engine, a term, a solution
- Methods to solve goals, ask for other solutions, navigate terms
- `alice.tuprolog`

Main Classes

- `Prolog`: a "Prolog Virtual Machine"
- `Theory`: a Prolog DB, as a text or list of clauses
- `SolveInfo`: the outcome of a goal
- `Term`: the interface to any term

Direct goal solution

```java
package u12;

// libraryDependencies += "it.unibo.alice.tuprolog" % "tuprolog" % "3.3.0"
import alice.tuprolog.*;

public class Test1 {

    public static void main(String[] args) throws Exception {
        final Prolog engine = new Prolog();
        final SolveInfo info = engine.solve("append([1],[2,3],X).");
        System.out.println(info.getSolution());
        // "append([1],[2,3],[1,2,3])"
    }
}
```

Getting all solutions

```java
package u12;

import alice.tuprolog.*;

public class Test2 {

    public static void main(String[] args) throws Exception {

        final Prolog engine = new Prolog();
        SolveInfo info = engine.solve("append(X,Y,[1,2,3]).");
        while (info.isSuccess()){
            System.out.print("solution: "+info.getSolution()+" - ");
            System.out.println("bindings: X/"+info.getTerm("X")
                                    +" Y/"+info.getTerm("Y"));
            if (engine.hasOpenAlternatives()){
                info=engine.solveNext();
            } else {
                break;
            }
        }
        // solution: append([],[1,2,3],[1,2,3]) - bindings: X/[] Y/[1,2,3]
        // ..
        // solution: append([1,2,3],[],[1,2,3]) - bindings: X/[1,2,3] Y/[]
    }
}
```

Getting all solutions

```java
package u12;

import alice.tuprolog.*;

public class Test2 {

    public static void main(String[] args) throws Exception {

        final Prolog engine = new Prolog();
        SolveInfo info = engine.solve("append(X,Y,[1,2,3]).");
        while (info.isSuccess()){
            System.out.print("solution: "+info.getSolution()+" - ");
            System.out.println("bindings: X/"+info.getTerm("X")
                                    +" Y/"+info.getTerm("Y"));
            if (engine.hasOpenAlternatives()){
                info=engine.solveNext();
            } else {
                break;
            }
        }
        // solution: append([],[1,2,3],[1,2,3]) - bindings: X/[] Y/[1,2,3]
        // ..
        // solution: append([1,2,3],[],[1,2,3]) - bindings: X/[1,2,3] Y/[]
    }
}
```

The structure of Terms

`Term`, The root of the hierarchy

Methods:

- `getRenamedCopy`: Changes the name of variables
- `isAtom`: Is this a constant name or number?
- `isList`: Is this a list?
- `unify`: Does this unify with that?

Hierarchy

- `Term`: The root of the hierarchy
  - `Var`: A variable (with a name of anonymous)
  - `Number`: int, float, and so on
    - `Double`, `Float`, `Int`, `Long`
  - `Struct`: a compound term, but also a list

Creating and using Terms

```java
package u12;

import alice.tuprolog.*;

public class Test3 {

    public static void main(String[] args) throws Exception {
        final Prolog engine = new Prolog();
        // [1]
        final Term list1=engine.toTerm("[1]");
        // [2,3]
        final Term list2=new Struct(new Term[]{new Int(2),new Int(3)});
        // append([1],[2,3],X).
        final Term app=new Struct("append",list1,list2,new Var("X"));
        final SolveInfo info = engine.solve(app);
        System.out.println(info.getSolution());
        // "append([1],[2,3],[1,2,3])"
    }
}
```

Reading Terms

```
1  package u12;
2
3  import alice.tuprolog.*;
4
5  public class Test4 {
6
7      public static void main(String[] args) throws Exception {
8          final Prolog engine = new Prolog();
9          Term t=new Struct();
10         // t will be [0,1,2,...,9,10]
11         for (int i=10;i>=0;i--){
12             t=new Struct(new Int(i),t);
13         }
14         final Term app=new Struct("append",t,t,new Var("X"));
15         final SolveInfo info = engine.solve(app);
16         final Struct t2=(Struct)info.getTerm("X");
17         for(java.util.Iterator i=t2.listIterator();i.hasNext();){
18             System.out.print(" "+i.next()); // 0 1 2 .. 10 0 1 2 .. 10
19         }
20     }
21 }
```

Setting a theory

```java
package u12;

import alice.tuprolog.*;

public class Test5 {

    public static void main(String[] args) throws Exception {

        final Prolog engine = new Prolog();
        final Theory t = new Theory(
                "search(E,[E|_]). " + // a space is needed here!
                "search(E,[_|L]):-search(E,L)."
        );
        // engine.setTheory(new FileInputStream("file.pl"));
        engine.setTheory(t);
        final SolveInfo info = engine.solve("search(1,[1,2,3]).");
        System.out.println("" + info.getSolution()); // search(1,[1,2,3])
    }
}
```

Which applications?

- Each time you want in Java something that is better implemented in Prolog

Example

Realise permutations over Lists

- Permutation are easy implemented in Prolog by a predicate `perm(+L,-L2)`
- Build a Java method that takes a `LinkedList` and returns a `Iterator<java.util.LinkedList>`, each time invoking `solveNext()` and building the `LinkedList` from the result.

Outline

1. On Java-Scala-Prolog Integration
2. Prolog calling Java
3. Java calling Prolog
4. Prolog libraries written in Java
5. Scala using Prolog

3) Prolog using Java: encapsulation approach

Library-oriented integration

- Prolog using Java: `->/2`, `new_object/3`, `returns/2`
- Java using Prolog: `Term` hierarchy, `Engine` class

Encapsulation-oriented integration

- Prolog using Java: a Java class becomes Prolog library
- Scala using Prolog: a Prolog theory embedded into a Scala program

Prolog Libraries

What is a Prolog Library?

- It is basically a Prolog DB
- It includes the definition of many predicates
- It can be loaded in different ways:
  - From another theory through goal `load_library/1`
  - In the 2P IDE through the library manager

Built-in examples

- `BasicLibrary`: Standard operators, e.g. `append`.
- `ISOLibrary`: Standard operators of ISO standard
- `IOLibrary`: for Input/Output, e.g. `tell`, `told`
- They are all automatically added by 2P IDE

Building a Library in Java

It is a very elegant approach

- Write a Java class where each method defines a different predicate
- Compile this class, e.g. with name `lib.mylibrary`
- Load the library with `load_library('lib.mylibrary')`

Structure of methods

- `public boolean <pred name>_<N>(T1 arg1,...,Tn argN)`
- `public Term <eval name>_<N>(T1 arg1,...,Tn argN)`
- where `Ti` is either `Term`, `Var`, etc.

A simple library

```
// run with: java -cp <path-to-libraries>:2p.jar alice.tuprologx.ide.GUILauncher
// be sure the library is compiled with Java 8
public class TestLibrary extends Library {

    private int termAsInt(Term t){
        return ((Number)t).intValue();
    }

    // try with: X is sum(10,20).
    public Term sum_2(Term arg0, Term arg1){
        return new Int(termAsInt(arg0)+termAsInt(arg1));
    }

    // try with: X is minus(20,10).
    public Term minus_2(Term arg0, Term arg1){
        return new Int(termAsInt(arg0)-termAsInt(arg1));
    }

    // try with: sum(Y,8,21).
    public boolean sum_3(Term arg0, Term arg1, Term arg){
        try {
            if (arg instanceof Var) {
                return arg.unify(this.getEngine(), sum_2(arg0, arg1));
            }
            if (arg0 instanceof Var) {
                return arg0.unify(this.getEngine(), minus_2(arg, arg1));
            }
            return arg1.unify(this.getEngine(), minus_2(arg, arg0));
        } catch (Exception e){
            return false;
        }
```

Outline

1. On Java-Scala-Prolog Integration
2. Prolog calling Java
3. Java calling Prolog
4. Prolog libraries written in Java
5. Scala using Prolog

Scala using Prolog: encapsulation approach

Library-oriented integration

- Prolog using Java: `->/2`, `new_object/3`, `returns/2`
- Java using Prolog: `Term` hierarchy, `Engine` class

Encapsulation-oriented integration

- Prolog using Java: a Java class becomes Prolog library
- Scala using Prolog: a Prolog theory embedded into a Scala program

Scala using Prolog (1/2)

```scala
object TryScala2P extends App:
  import Scala2P.{*, given}

  val engine: Term => LazyList[Term] = mkPrologEngine("""
    member([H|T],H,T).
    member([H|T],E,[H|T2]):- member(T,E,T2).
    permutation([],[]).
    permutation(L,[H|TP]) :- member(L,H,T), permutation(T,TP).
  """)
  /*
  engine("permutation([1,2,3],L)") foreach (println(_))
  // permutation([1,2,3],[1,2,3]) ... permutation([1,2,3],[3,2,1])
  */

  val input = Struct("permutation",(1 to 20), Var())
  engine(input) map (extractTerm(_,1)) foreach (println(_))
  // [1,2,3,4,..,20] ... [1,2,..,15,20,16,18,19,17]
```

Scala using Prolog (2/2)

```scala
// with dependency: "it.unibo.alice.tuprolog" % "tuprolog" % "3.3.0"
import alice.tuprolog.*

object Scala2P:
  given Conversion[String, Term] = Term.createTerm(_)
  given Conversion[Seq[_], Term] = _.mkString("[",",","]")

  def extractTerm(t:Term, i:Integer): Term =
    t.asInstanceOf[Struct].getArg(i).getTerm

  def mkPrologEngine(clauses: String*): Term => LazyList[Term] =
    val engine = Prolog()
    engine.setTheory(Theory(clauses mkString " "))
    goal => new Iterable[Term]{
      override def iterator = new Iterator[Term] {
        var solution = engine.solve(goal);
        override def hasNext =
          solution.isSuccess || solution.hasOpenAlternatives
        override def next() =
          try { solution.getSolution} finally {solution = engine.solveNext }
      }
    }.to(LazyList)
```

Preview: a functional/monadic approach to Logic Programming

```
1
2   // lookup([H|T],H,zero,T).
3   // lookup([H|T],E,s(N),[H|T2]):- lookup(T,E,N,T2).
4
5   def lookup[A](list : List[A]): LazyList[(A,Int,List[A])] = list match
6     case Nil => LazyList()
7     case h :: t =>
8       (h,0,t) #::
9       (for (e,n,t2) <- lookup(t) yield (e,n+1,h::t2))
10
11  println( lookup( List(10,20,30,20) ).toList)
12  // List((10,zero,List(20, 30, 20)), (20,s(zero),List(10, 30, 20)), ...
13
14  println( lookup( List(10,20,30,20) ).collect{case (20,n,_) => n}.toList)
15  // List(s(zero), s(s(s(zero))))
16
17  println( lookup( List(10,20,30,40) ).collect{case (e,3,_) => e}.toList)
18  // List(40)
```